VNU-HCM

University of Science

faculty of information technology

Introduction to computer science

Project: Go Game

Courses: CS160

Students:

Anh, Tu Hoang (25125005)

Huy, Tran Quoc (25125053)

November 24, 2025

November 24, 2025

Contents

1 Overview **3**

1.1 Project Description . . . 3

1.2 Project Objectives . . . 3

2 Requirements Specification **3**

2.1 Game Models . . . 3

2.1.1 Board Sizes . . . 3

2.1.2 UI Requirements . . . 3

2.2 Board Representation . . . 4

2.3 Movements . . . 5

2.3.1 Stone Placement . . . 5

2.3.2 Move Validity . . . 5

2.3.3 Capturing Logic, Game End Conditions . . . 7

2.4 Game State Management . . . 9

2.4.1 Undo Move . . . 9

2.4.2 Redo Move . . . 10

2.4.3 Skip move . . . 10

2.4.4 Reset Game (Future Feature) . . . 11

2.5 Save and Load . . . 11

3 Design **13**

3.1 Architecture Diagram . . . 13

3.2 Design Decisions . . . 13

4 Implementation Details **13**
4.1 Code Organization . . . 13
4.1.1 Game Logic . . . 14
4.1.2 Set up Visual Studio . . . 14
4.2 Testing and Deployment . . . 15
4.2.1 Strategies . . . 15
4.2.2 Results . . . 15
4.2.3 Issues or Limitations . . . 16
4.2.4 Deployment . . . 16

5 Future Improvements **17**
5.1 Planned Features . . . 17
5.2 Technical Improvements . . . 17
5.3 AI Enhancement . . . 17

6 Work Division **18**
6.1 Team Members . . . 18
6.2 Responsibilities . . . 18
6.3 Collaboration Tools . . . 18

7 Tutorial **19**
7.1 Installation . . . 19
7.2 How to Play . . . 19
7.3 Saving and Loading . . . 24

8 Conclusion **24**
8.1 Project Summary . . . 24
8.2 Technical Achievements . . . 25
8.3 Learning Outcomes . . . 25

9 References **26**

1 Overview

1.1 Project Description

This project implements a fully functional digital version of Go (Weiqi/Baduk), one of the oldest and most complex board games in the world. The implementation uses C++ as the primary programming language and Raylib as the graphics library, providing an intuitive graphical interface with support for two-player modes.

1.2 Project Objectives

1. Implement complete Go game rules including stone placement, capture mechanics, and Ko rule
2. Develop an intuitive graphical user interface with responsive design
3. Support multiple board sizes (11×11, 13×13, 19×19)
4. Provide game state management (save/load functionality)

2 Requirements Specification

2.1 Game Models

2.1.1 Board Sizes

The implementation supports three standard Go board sizes:

- **11×11**: Suitable for beginners, faster games
- **13×13**: Intermediate level, balanced gameplay
- **19×19**: Professional standard, complex strategy

2.1.2 UI Requirements

The user interface must provide:

- Clear visual representation of the game board
- Stone placement indicators on hover
- Turn indicator (Black/White player)
- Game control buttons (Undo, Redo, Skip, Exit)
- Menu system for game configuration

2.2 Board Representation

The game board is represented using a vector-based system defined in `Board.h`:

```
class point {
public:
    int x, y;
    bool black;
    point(int x_ = 0, int y_ = 0, bool black_ = 0)
        : x(x_), y(y_), black(black_) {}
};

class Board {
public:
    int X[4] = {1, -1, 0, 0};
    int Y[4] = {0, 0, 1, -1};

    std::vector<std::pair<int, int>> ListOfCapturedStone(int x, int y,
   int turn);
    void RemoveCapturedStones(int x, int y, int turn,
                              std::vector<point> &points, int &is_captured
   );
    bool ValidMove(int x, int y, int turn);
    void PlaceStone(int i, int j);

    int turn = 0, maxturn = 0;
```

```
    int black = 0;
    bool ExitToMenu = false;
    std::vector<point> points;
    soundEffect sounds;
    IMG img;
};
```

Listing 1: Board Data Structure from Board.h

2.3 Movements

2.3.1 Stone Placement

Players place stones on the intersections of the board grid. The implementation validates moves based on:

- Empty intersection check
- Suicide rule prevention
- Ko rule enforcement

2.3.2 Move Validity

The `ValidMove` function from `Board.cpp` ensures legal moves:

```
bool Board::ValidMove(int x, int y, int black) {
    for (auto &p : points) {
        if (p.x == x && p.y == y) {
            return false; // position already occupied
        }
    }

    std::vector<point> temp_points = points;
    temp_points.push_back({x, y, (bool)black});
    std::vector<std::pair<int, int>> captured = ListOfCapturedStone(x, y,
    black);
```

```

    int is_captured = 0;
    RemoveCapturedStones(x, y, black, temp_points, is_captured);

    if (turn >= 1) {
        // Check for Ko rule violation
        if (isTheSame(temp_points, stage[turn - 1])) {
            return false; // Ko rule violation
        }
    }

    if (!captured.empty()) {
        return true; // capturing move is valid
    }


    // Check if the new stone has any liberties
    std::vector<std::vector<int>> board(gridSize, std::vector<int>(
gridSize, -1));
    for (const auto &p : temp_points) {
        board[p.x][p.y] = p.black;
    }

    std::queue <std::pair<int, int>> q;
    q.push({x, y});
    board[x][y] = -2; // mark as visited

    while (!q.empty()) {
        int x = q.front().first;
        int y = q.front().second;
        q.pop();
        for (int dir = 0; dir < 4; dir++) {
            int nx = x + X[dir];
```

```
            int ny = y + Y[dir];
            if (nx >= 0 && nx < gridSize && ny >= 0 && ny < gridSize) {
                if (board[nx][ny] == -1) {
                    return true; // has liberty
                } else if (board[nx][ny] == black) {
                    q.push({nx, ny});
                    board[nx][ny] = -2; // mark as visited
                }
            }
        }
    }

    return false; // no liberties found
}
```

Listing 2: Move Validation Logic

2.3.3 Capturing Logic, Game End Conditions

Stones are captured when completely surrounded by opponent stones. The capture detection algorithm uses flood-fill to identify connected groups and check for liberties:

```
std::vector<std::pair<int, int>> Board::ListOfCapturedStone(int x, int y,
    int turn) {
    std::vector<std::vector<int>> board(19, std::vector<int>(19, -1));
    std::vector<std::vector<int>> degree(19, std::vector<int>(19, 0));

    for (const auto &p : points) {
        board[p.x][p.y] = p.black;
    }

    board[x][y] = turn;
    for (int x = 0; x < 19; x++) {
        for (int y = 0; y < 19; y++) {
```

```
            if (board[x][y] != -1) {
                for (int dir = 0; dir < 4; dir++) {
                    int nx = x + X[dir];
                    int ny = y + Y[dir];
                    if (nx >= 0 && nx < 19 && ny >= 0 && ny < 19) {
                        if (board[nx][ny] == -1) {
                            degree[x][y]++;
                        }
                    }
                }
            }
        }
    }

    std::vector<std::pair<int, int>> Remove;

    for (int x = 0; x < 19; x++) {
        for (int y = 0; y < 19; y++) {
            if (board[x][y] == (1 - turn) && degree[x][y] == 0) {
                std::queue <std::pair<int, int>> q;
                q.push({x, y});
                std::vector<std::pair<int, int>> toRemove;
                bool liberty = 1;
                board[x][y] = -2; // mark as visited

                while (!q.empty()) {
                    int cx = q.front().first;
                    int cy = q.front().second;
                    q.pop();
                    toRemove.push_back({cx, cy});

                    for (int dir = 0; dir < 4; dir++) {
                        int nx = cx + X[dir];
```

```
                        int ny = cy + Y[dir];
                        if (nx >= 0 && nx < 19 && ny >= 0 && ny < 19) {
                            if (board[nx][ny] == (1 - turn)) {
                                if (degree[nx][ny] > 0)
                                    liberty = 0;

                                board[nx][ny] = -2; // mark as visited
                                q.push({nx, ny});
                            }
                        }
                    }
                }

                if (liberty) {
                    for (auto stone : toRemove) {
                        Remove.push_back(stone);
                    }
                }
            }
        }
    }

    return Remove;
}
```

Listing 3: Capture Detection Algorithm

2.4 Game State Management

2.4.1 Undo Move

Players can undo moves to review strategy. Implementation stores board states:

```
void Board::PressUndoButton() {
    PlaySound(sounds.press);
```

```
    if (turn > 0) {
        turn--;
        points = stage[turn];
        black = StageTurn[turn];
    }
}
```

Listing 4: Undo Functionality from Board.cpp

2.4.2 Redo Move

Allows players to redo previously undone moves:

```
void Board::PressRedoButton() {
    turn = maxturn;
    points = stage[turn];
    black = StageTurn[turn];
    PlaySound(sounds.press);
}
```

Listing 5: Redo Functionality

2.4.3 Skip move

Players can pass their turn, which is standard in Go:

```
void Board::PressSkipButton() {
    PlaySound(sounds.press);
    turn++;
    black ^= 1;
    stage[turn] = points;
    StageTurn[turn] = black;
    isSkip[turn] = 1;
    if (isSkip[turn - 1]) {
        matchEnd = 1;
    }
```

```
}
```

Listing 6: Skip Turn Implementation

2.4.4 Reset Game (Future Feature)

The reset functionality allows starting a new game while preserving configuration.

2.5 Save and Load

Game state persistence is implemented through file I/O operations defined in `SaveGame.cpp` and `LoadGame.cpp`:

```
void SaveGame(const Board& board, std::string filename) {
    std::ofstream out(filename);
    if (!out.is_open()) return;

    out << board.turn << std::endl;
    out << board.black << std::endl;
    out << board.points.size() << std::endl;

    for (auto p : board.points) {
        out << p.x << " " << p.y << " " << p.black << std::endl;
    }

    out.close();
}
```

Listing 7: Save Game Implementation from SaveGame.cpp

```
Board LoadGame(std::string filename) {
    Board board;
    std::ifstream in(filename);
    if (!in.is_open()) return board;

    int turn, black, size;
```

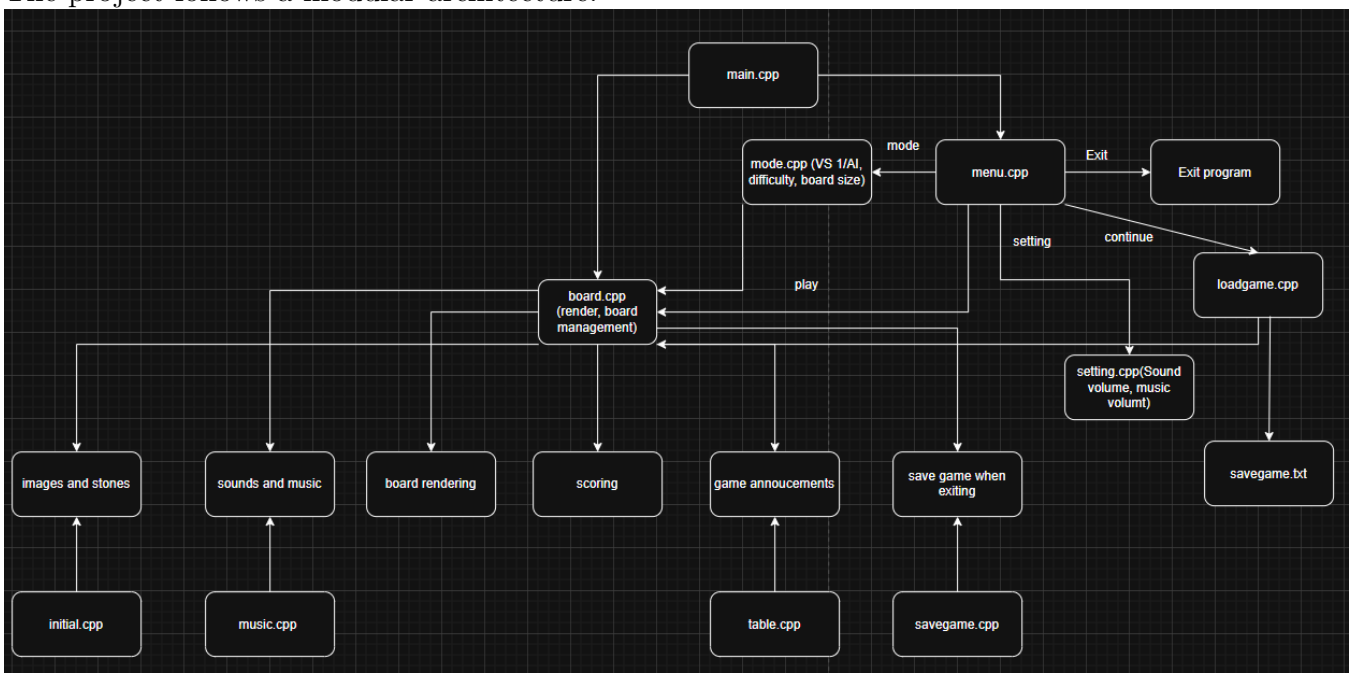
```
    in >> turn >> black >> size;

    board.turn = turn;
    board.maxturn = turn;
    board.black = black;

    for (int i = 0; i < size; i++) {
        int x, y, b;
        in >> x >> y >> b;
        board.points.push_back({x, y, (bool)b});
    }

    in.close();
    return board;
}
```

Listing 8: Load Game Implementation from LoadGame.cpp

3 Design

3.1 Architecture Diagram

The project follows a modular architecture:



3.2 Design Decisions

- **letterboxing technique**: The game uses a virtual resolution approach with letterboxing to ensure consistent appearance across different screen sizes.
- **Graphics Seperation**: UI elements are seperated from the Game Logic for parallel development and better maintainability.
- **Game Logic**: Game state and movement based on vector, requires less calculation.

4 Implementation Details

4.1 Code Organization

The project is organized into logical modules with clear responsibilities:

Module	Responsibility
Board.cpp/h	Core game logic, move validation, capture detection
Button.cpp/h	UI button system with hover detection and callbacks
Menu.cpp/h	Main menu interface and navigation
Mode.cpp/h	Game mode selection (1v1, vs AI, board size)
IMG.cpp/h	Image loading and texture management
soundEffect.cpp/h	Sound effect management
Music.cpp/h	Background music playback
SaveGame.cpp/h	Game state serialization
LoadGame.cpp/h	Game state deserialization
Setting.cpp/h	Game settings (volume controls)
font.cpp/h	Font rendering utilities
initial.cpp/h	Global configuration and initialization

Table 1: Module Organization

4.1.1 Game Logic

The core game logic handles:

- Stone placement validation
- Capture detection using flood-fill algorithm
- Ko rule enforcement
- Suicide prevention
- Turn management

4.1.2 Set up Visual Studio

The project uses CMake for build configuration. Key setup from `CMakeLists.txt`:

```
cmake_minimum_required (VERSION 3.16)

project("Project")

add_definitions(-DWIN32_LEAN_AND_MEAN -DNOMINMAX)

```

```
#Automatically pick up files added to src
file(GLOB_RECURSE HEADER_FILES ${CMAKE_CURRENT_SOURCE_DIR}/src/*.h)
file(GLOB_RECURSE SOURCE_FILES ${CMAKE_CURRENT_SOURCE_DIR}/src/*.cpp)

add_executable(Project ${HEADER_FILES} ${SOURCE_FILES})

target_include_directories(Project PRIVATE
  ${CMAKE_SOURCE_DIR}/include
)

add_subdirectory("raylib")

target_link_libraries(${PROJECT_NAME} PUBLIC raylib)
```

Listing 9: CMake Build Configuration

4.2 Testing and Deployment

4.2.1 Strategies

Testing approach includes:

1. **Unit Testing**: Individual module validation
2. **Integration Testing**: Component interaction verification
3. **Gameplay Testing**: Rule compliance and edge case handling
4. **UI Testing**: Responsiveness and visual consistency
5. **Performance Testing**: Frame rate stability and memory usage

4.2.2 Results

Test results summary:

Test Category	Pass	Fail
Stone Placement	OK	-
Capture Detection	OK	-
Ko Rule Enforcement	OK	-
Undo/Redo Functionality	OK	-
Save/Load Game State	OK	-
UI Responsiveness	OK	-
Resolution Scaling	OK	-
Audio Playback	OK	-

Table 2: Testing Results

4.2.3 Issues or Limitations

Known limitations:

- AI implementation is placeholder (future enhancement)
- No network multiplayer support
- Limited to three board sizes
- Manual scoring required at game end

4.2.4 Deployment

Build instructions for Windows:

```
1. Install Visual Studio 2022 with C++ development tools
2. Install CMake (version 3.25 or higher)
3. Clone the repository
4. Open CMakeLists.txt in Visual Studio
5. Configure CMake settings
6. Build the solution (Ctrl+Shift+B)
7. Run the executable from build/Debug or build/Release
```

For command-line build:

```
mkdir build
cd build
```

```
cmake ..
cmake --build . --config Release
.\Release\Project.exe
```

5 Future Improvements

5.1 Planned Features

- Implement AI to play against player
- Add game replay functionality
- Include tutorial mode for beginners

5.2 Technical Improvements

Code quality enhancements:

- Refactor button system for better modularity
- Implement observer pattern for game events
- Add comprehensive unit test suite
- Optimize rendering pipeline
- Improve memory management

5.3 AI Enhancement

Future AI implementation plan:

1. **Basic AI**: Random valid moves
2. **Pattern Recognition**: Common joseki patterns
3. **Minimax Algorithm**: Simple evaluation function

6 Work Division

6.1 Team Members

This project was developed by:

- Hoang Anh - Developer
- Quoc Huy - Designer

6.2 Responsibilities

Project responsibilities breakdown:

Component	**Developer**
Core Game Logic	Hoang Anh
UI System	Quoc Huy
Menu System	Hoang Anh
Save/Load System	Hoang Anh
Audio Integration	Hoang Anh
Graphics	Quoc Huy
Documentation	Quoc Huy
Testing	Quoc Huy

Table 3: Work Division

6.3 Collaboration Tools

Development tools used:

- **Version Control**: Git
- **IDE**: Visual Studio Code, Visual Studio 2022
- **Build System**: CMake
- **Documentation**:Overleaf

7 Tutorial

7.1 Installation

System Requirements:

- Windows 7 or later (64-bit)
- 1GB RAM minimum
- OpenGL 3.3 compatible graphics card
- 100MB free disk space

Installation Steps:

1. Download the release package
2. Extract to desired location
3. Run the game

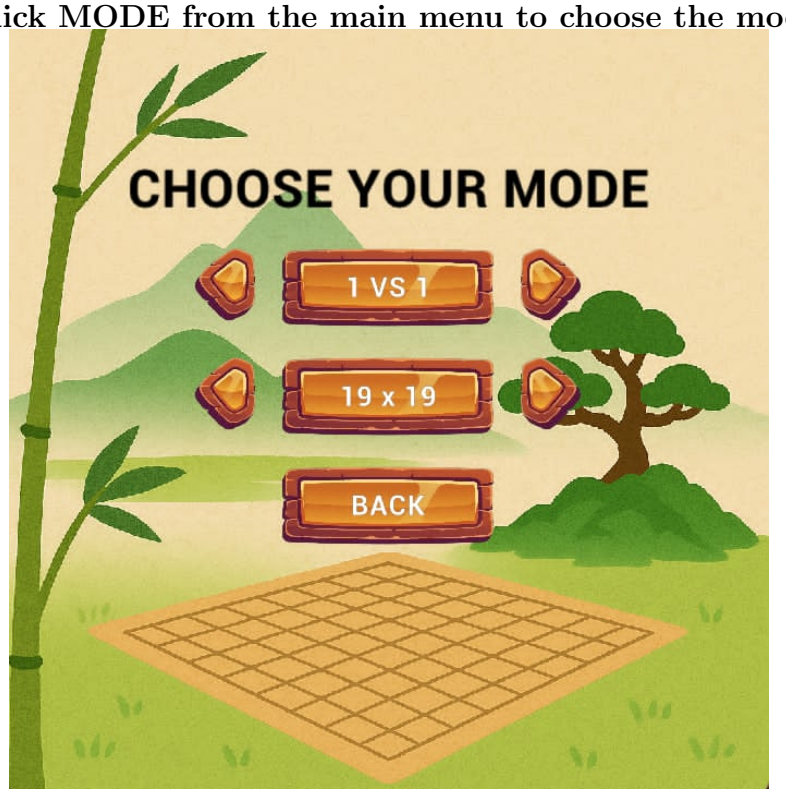
7.2 How to Play

Starting a Game:

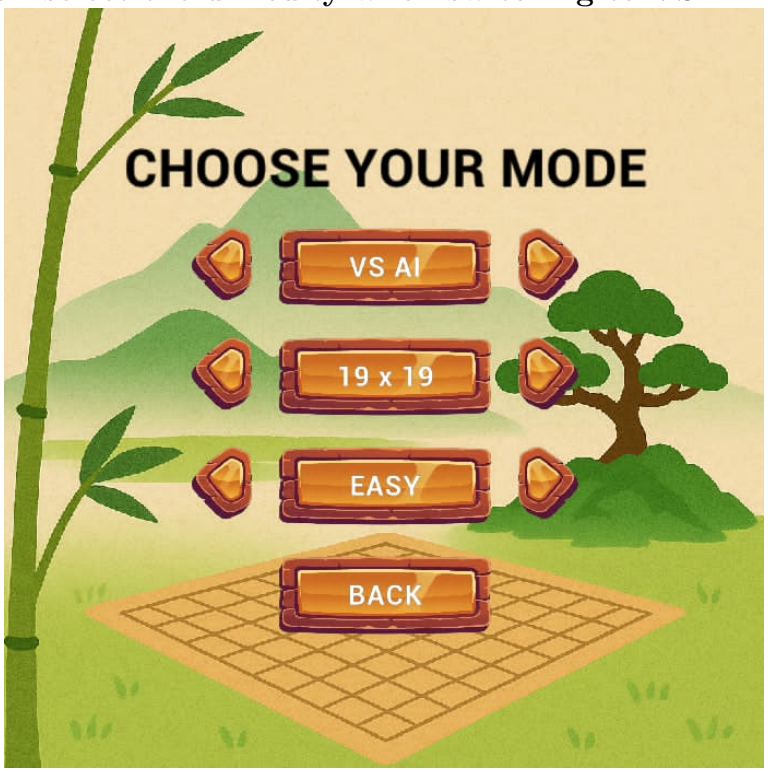
1. **Launch the application**

GO BOARD
CONTINUE
PLAY
MODE
SETTING
EXIT

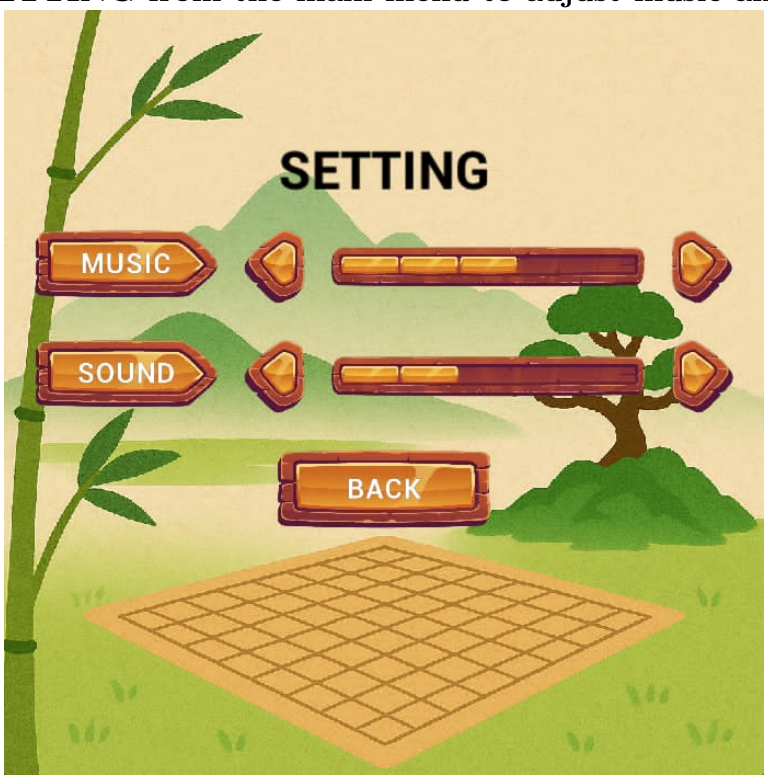
2. **Click MODE from the main menu to choose the mode**



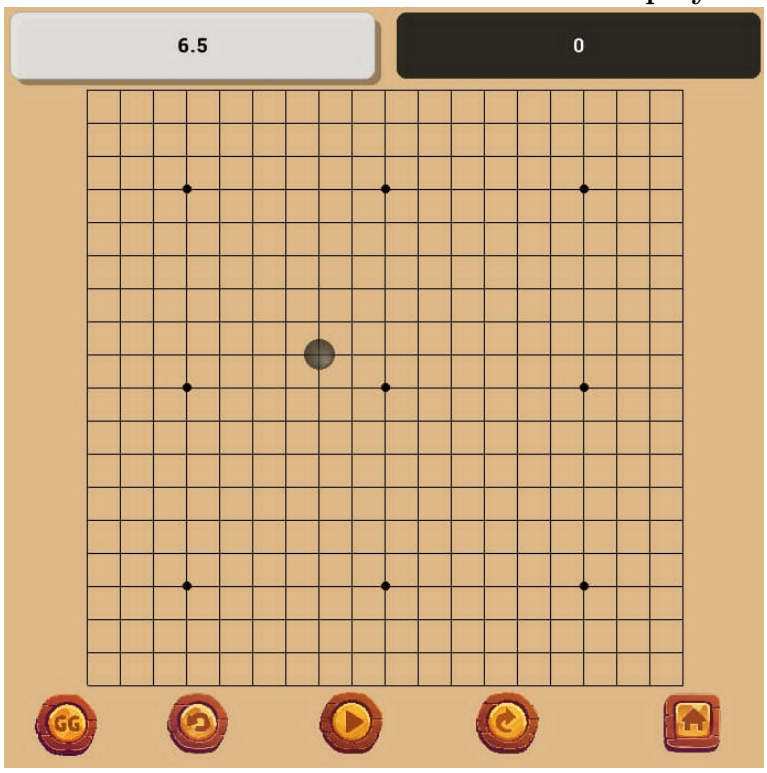
3. **You can select the difficulty when switching to VS AI MODE**



4. **Click SETTING from the main menu to adjust music and sound**



5. **Click PLAY from the main menu to play**



6. **End game will pop up when the match is over**

Game Controls:

- **Left Click**: Place stone at intersection
- **Undo Button**: Take back the last move
- **Redo Button**: Redo undone move
- **Skip Button**: Pass turn
- **Exit Button**: Return to main menu
- **GG Button**: Resign game and view results

Rules of Go:

1. Players alternate placing stones on empty intersections
2. Stones are captured when completely surrounded
3. The Ko rule prevents immediate recapture
4. Suicide moves are not allowed
5. Game ends when both players pass consecutively
6. Player with more points wins

7.3 Saving and Loading

- **Auto-Save** : The game automatically saves when you exit to menu. Your progress is stored in SaveGame.txt.
- **Loading Game** : Click "CONTINUE" from the main menu to resume your last saved game.

8 Conclusion

8.1 Project Summary

This project successfully implements a fully functional Go game with the following achievements:

- Complete implementation of Go rules including capture, Ko, and suicide prevention

- Responsive UI with resolution-independent rendering
- Multiple board sizes for different skill levels
- Save/load functionality for game continuity
- Clean, modular code architecture

8.2 Technical Achievements

Key technical accomplishments:

- Efficient flood-fill algorithm for capture detection
- Virtual resolution system with letterboxing
- State management pattern for undo/redo
- File I/O for game persistence
- Custom UI component system

8.3 Learning Outcomes

Skills developed during this project:

- C++ programming and OOP design patterns
- Game development fundamentals
- Graphics programming with Raylib
- Algorithm implementation (graph traversal, state management)
- Build system configuration with CMake

9 References

References

[1] Raylib - A simple and easy-to-use library to enjoy videogames programming. *https://www.raylib.com/*

[2] American Go Association. *The Rules of Go*. https://www.usgo.org/rules

[3] CMake Documentation. *https://cmake.org/documentation/*

[4] Background Music *https://freetouse.com/music/category/relaxing*

[5] Sound Effects *https://www.epidemicsound.com/sound-effects/categories/cartoon/*

[6] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. *Introduction to Algorithms* (3rd ed.). MIT Press.